



基于深度学习的开源软件代码的 漏洞检测系统

软件学院

目录 | contents



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

01

项目介绍

Northwestern Polytechnical University

02

研究过程

Northwestern Polytechnical University

03

代码实例

Northwestern Polytechnical University

04

未来展望

Northwestern Polytechnical University





西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

01

项目介绍

S t r o n g P r e p a r a t i o n

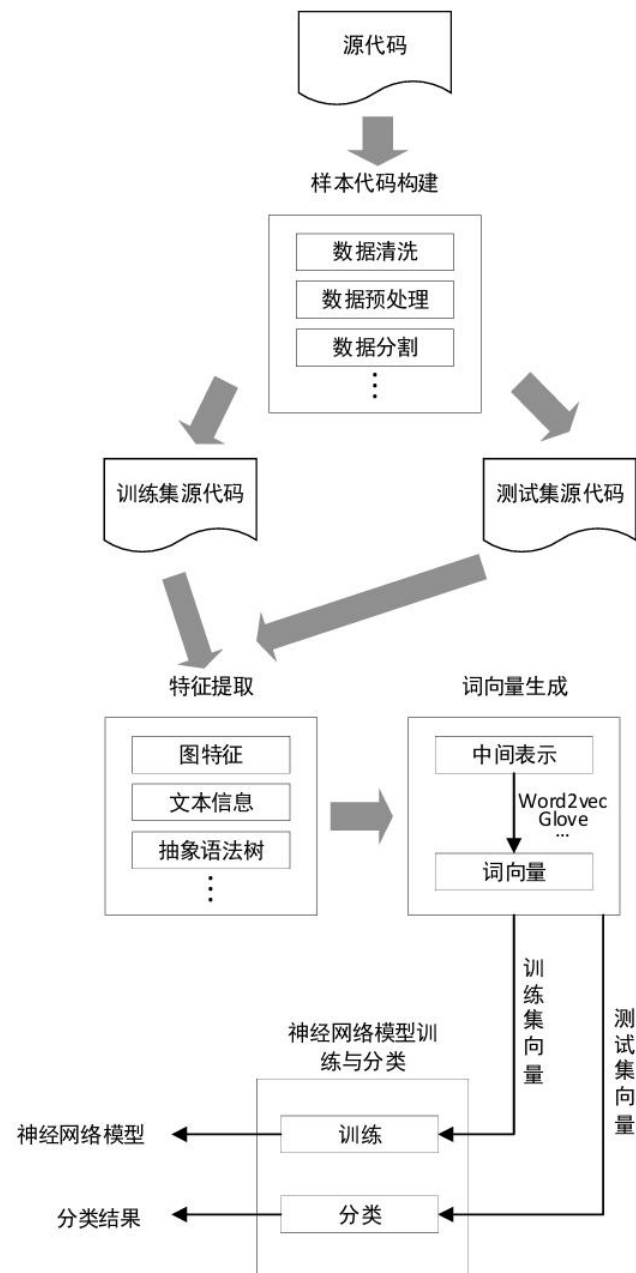
本项目聚焦于构建一套基于深度学习的开源代码漏洞检测系统，旨在通过智能化手段提升软件安全性。研究内容围绕以下核心环节展开：

1. 多样化数据集构建

- 系统性收集涵盖C/C++、Java、Python等主流语言的源代码样本，包含已验证的漏洞代码（如CVE公开漏洞案例）与安全代码，构建平衡且具有代表性的训练数据集。
- 通过数据清洗、标注及增强技术（如代码片段变异、上下文扩展），提升数据集对复杂漏洞模式的覆盖能力。

2. 代码特征深度解析

- 采用卷积神经网络（CNN）捕捉代码的局部语法特征（如变量命名、函数调用），结合循环神经网络（RNN）建模代码上下文依赖关系（如循环、条件分支）。
- 融合代码的抽象语法树（AST）、控制流图（CFG）等结构化表示，从语义层面识别潜在漏洞的异常模式（如内存越界、权限控制缺失）。

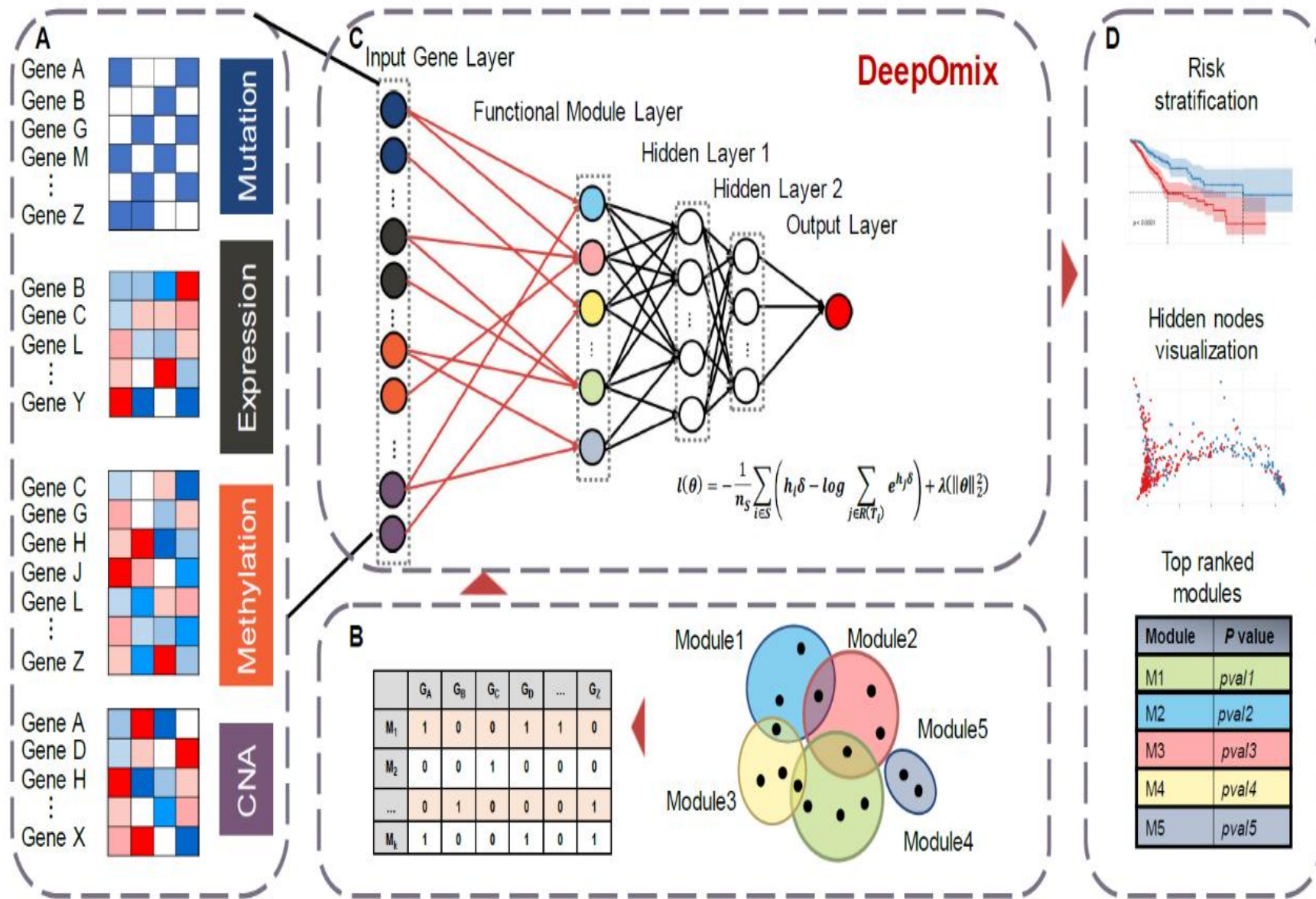


3. 高精度模型训练与优化

- 设计混合神经网络架构，整合 CNN、RNN与注意力机制，强化模型对长距离代码逻辑的关联分析能力。

- 通过对抗训练、动态权重调整等技术，提升漏洞检出率。

本系统不仅为开发者提供“即插即用”的自动化代码审查工具，还可通过可视化报告定位风险代码位置，并推荐修复方案，助力实现“开发即安全”的软件生命周期管理。研究成果将显著降低企业安全维护成本，为开源生态提供可信赖的防护屏障。





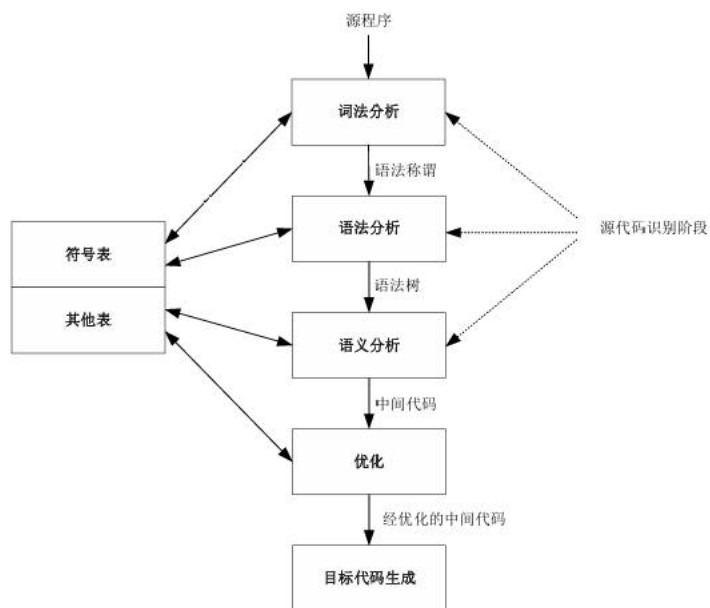
西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

02

研究过程

S t r o n g P r e p a r a t i o n

首先，我们学习到了编译器的工作流程以及对实际语句的识别



编译器工作流程

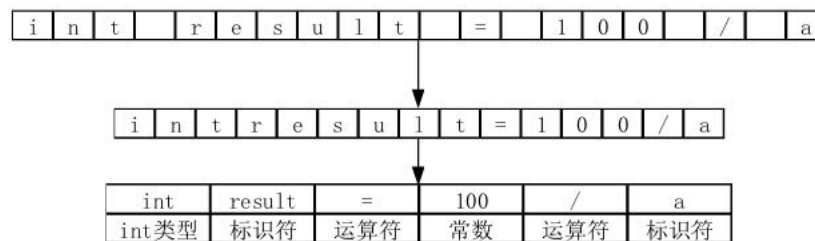
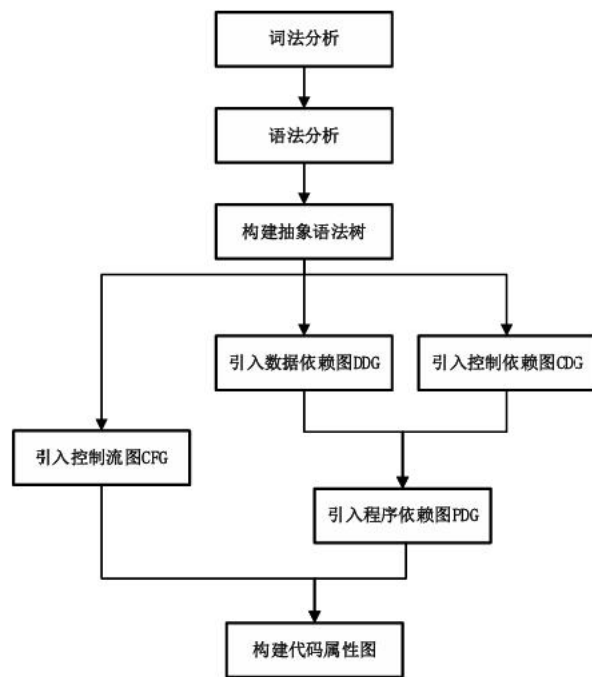


图 2-2 实际语句的识别过程

实际语句的识别

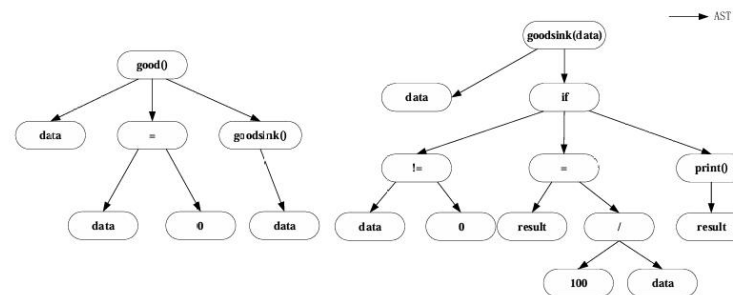
其次，我们根据代码属性图分析技术方案并用语法树分析代码



代码属性图

表 3-1 示例代码

```
1: void good(){
2:   int data;
3:   data = 0;
4:   goodsink(data);
5: }
6: void goodsink(int data){
7:   if(data != 0){
8:     int result = 100 / data;
9:     print(result);
10:  }
11: }
```



AST语法树

然后，学习对应的CFG属性图和PDG属性图

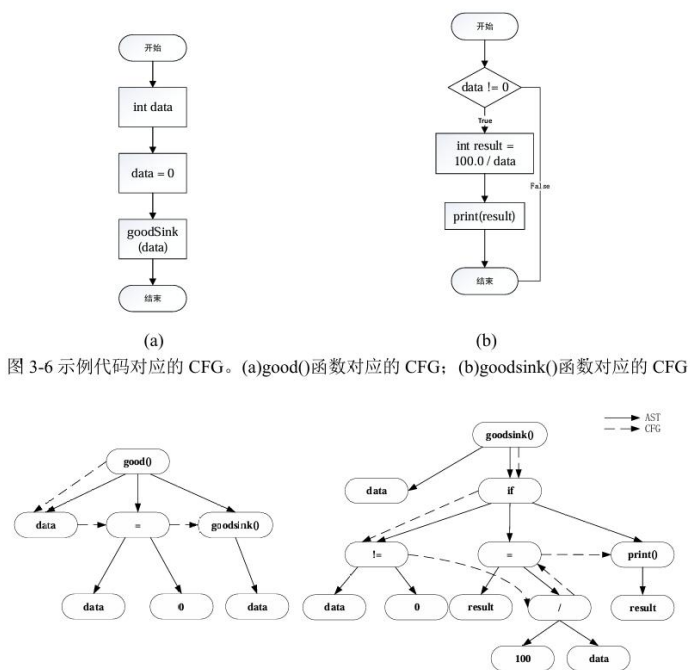


图 3-6 示例代码对应的 CFG。(a)good()函数对应的 CFG; (b)goodsink()函数对应的 CFG

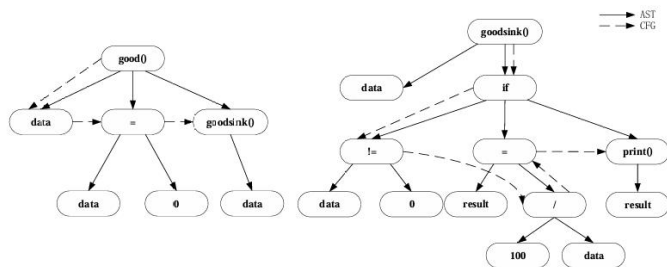


图 3-7 示例代码对应的 CFG 属性图

CFG属性图

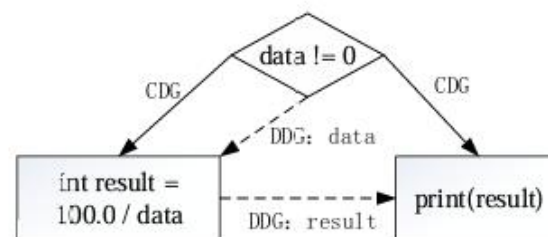


图 3-8 示例代码中 goodSink()对应的 PDG

PDG属性图

最后，学习对漏洞检测机理和邻接矩阵

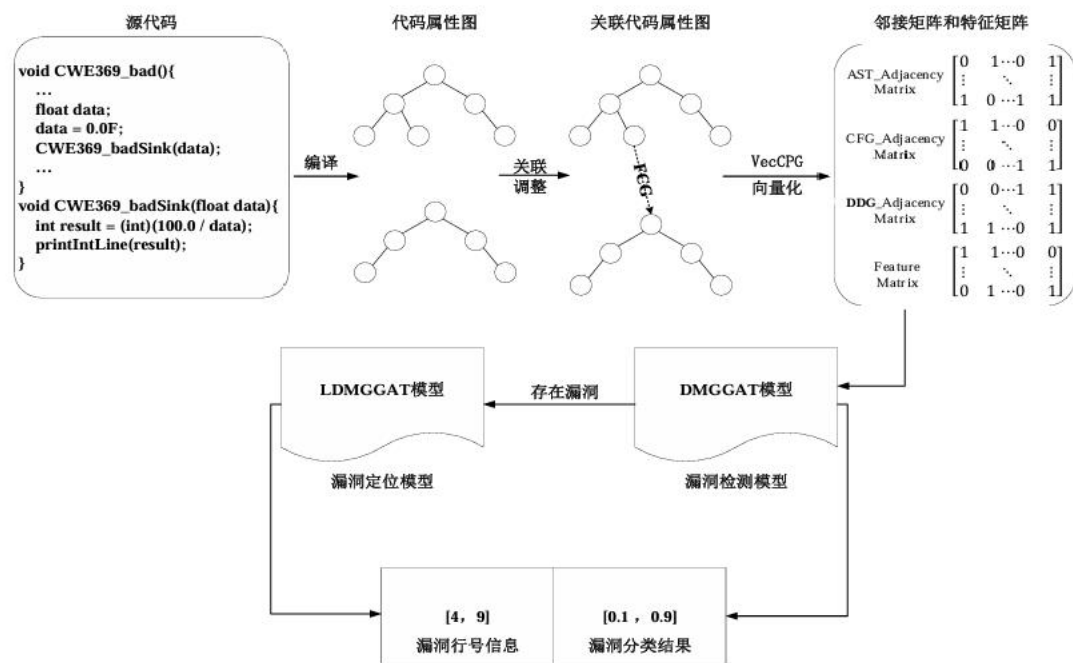


图 4-4 源代码漏洞检测框架图

漏洞检测机理

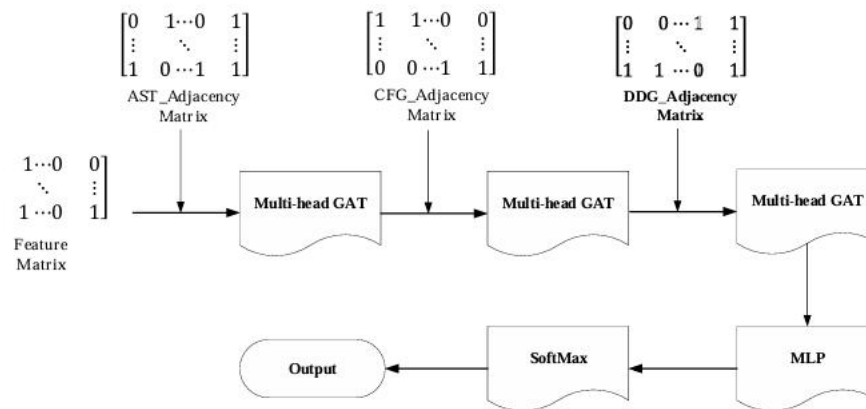


图 4-6 DMGGAT 结构

邻接矩阵



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

03

代码实例

S t r o n g P r e p a r a t i o n

板块一：代码到图结构的转换

```
python
def _extract_ast(self, code):
    """将代码解析为AST"""
    tree = self.parser.parse(bytes(code, 'utf8'))
    root_node = tree.root_node

    # 将AST转换为NetworkX图
    G = nx.DiGraph()

    def traverse_node(node, parent_id=None):
        node_id = len(G)
        # 提取节点类型和文本作为节点特征
        G.add_node(node_id,
                   type=node.type,
                   text=code[node.start_byte:node.end_byte],
                   start=node.start_byte,
                   end=node.end_byte)

        if parent_id is not None:
            G.add_edge(parent_id, node_id)

        for child in node.children:
            traverse_node(child, node_id)

    traverse_node(root_node)
    return G
```

功能介绍：

这个函数是系统的基础，它实现了源代码到图结构的转换过程：

利用Tree-sitter解析器将源代码解析为抽象语法树(AST)通过深度优先遍历，将AST中的每个节点和边转换为图数据结构

保留代码的结构信息和语义元素，如节点类型和文本内容创建了代码的结构化表示，使机器学习模型能够处理代码

板块二：GNN模型架构

python

```
class VulnerabilityGNN(nn.Module):
    def __init__(self, in_feats, h_feats, num_classes=2,
gnn_type='gcn'):
        super(VulnerabilityGNN, self).__init__()
        self.gnn_type = gnn_type

        # 第一层GNN
        if gnn_type == 'gcn':
            self.conv1 = GraphConv(in_feats, h_feats)
        elif gnn_type == 'gat':
            self.conv1 = GATConv(in_feats, h_feats,
num_heads=4)
        elif gnn_type == 'sage':
            self.conv1 = SAGEConv(in_feats, h_feats,
aggregator_type='mean')

        # 第二层GNN
        if gnn_type == 'gcn':
            self.conv2 = GraphConv(h_feats, h_feats)
        elif gnn_type == 'gat':
            self.conv2 = GATConv(h_feats * 4, h_feats,
num_heads=1)
        elif gnn_type == 'sage':
            self.conv2 = SAGEConv(h_feats, h_feats,
aggregator_type='mean')

        # 预测层
        self.predict = nn.Linear(h_feats, num_classes)
        self.dropout = nn.Dropout(0.5)
```

功能介绍：

这段代码定义了漏洞检测的核心图神经网络模型：

- 支持多种GNN变体（GCN、GAT、GraphSAGE），适应不同类型的代码结构
- 采用两层GNN架构进行消息传递和特征学习
- GAT变体使用注意力机制，自动学习不同节点之间的重要性权重
- Dropout层防止过拟合，提高模型泛化能力
- 最终的线性层将节点表示映射为漏洞预测结果

板块三：消息传递机制python

python

```
def forward(self, g, features):  
    # 第一层GNN消息传递  
    if self.gnn_type == 'gat':  
        h = self.conv1(g, features)  
        h = h.view(h.shape[0], -1) # 展平多头注意力的结果  
    else:  
        h = self.conv1(g, features)  
  
    h = F.relu(h)  
    h = self.dropout(h)  
  
    # 第二层GNN消息传递  
    h = self.conv2(g, h)  
    h = F.relu(h)  
    h = self.dropout(h)  
  
    # 预测  
    return self.predict(h)
```

功能介绍：

这个函数实现了GNN的核心消息传递过程：

通过图卷积操作，节点能够聚合来自相邻节点的信息

每个节点根据其邻居节点的特征更新自己的表示

多层消息传递使节点能够获取更广范围的上下文信息

激活函数引入非线性，增强模型表达能力

这种机制使模型能够理解代码元素之间的语义关系和依赖性，例如变量的使用和函数的调用关系

板块四：漏洞检测和解释python

python

```
def explain_prediction(self, code):
    # 解释模型对代码的预测
    result = self.predict(code, return_explanations=True)
    explanation = {
        'overall_vulnerability': result['vulnerability_probability'],
        'prediction': "Vulnerable" if result['is_vulnerable'] else "Not Vulnerable",
        'top_vulnerable_elements': []
    }
    # 提取最可能的漏洞代码元素
    for node in result['vulnerable_nodes']:
        explanation['top_vulnerable_elements'].append({
            'code': node['text'],
            'type': node['type'],
            'confidence': node['prob']
        })
    # 识别可能的漏洞类型
    vuln_types = set()
    dangerous_funcs = {'strcpy', 'strcat', 'gets', 'sprintf', 'system', 'exec'}

    for node in result['vulnerable_nodes']:
        node_text = node['text']
        # 检查危险函数
        for func in dangerous_funcs:
            if func in node_text:
                if 'str' in func:
                    vuln_types.add('Buffer Overflow')
                elif func in {'system', 'exec'}:
                    vuln_types.add('Command Injection')

    explanation['potential_vulnerability_types'] = list(vuln_types)

    return explanation
```

功能介绍：

这段代码展示了系统如何提供可解释的漏洞检测结果：不仅提供整体漏洞概率，还定位具体的漏洞代码元素通过分析节点特征和预测概率，识别最可能包含漏洞的代码片段

基于已知的漏洞模式库，推断潜在的漏洞类型（如缓冲区溢出、命令注入等）

为每个可疑代码元素提供置信度评分，帮助开发者优先修复高风险问题

这种可解释性设计使系统不仅能检测漏洞，还能辅助开发者理解和修复问题



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

04

未来展望

S t r o n g P r e p a r a t i o n

本项目作为一项聚焦代码安全的半成品研究，未来将围绕技术深化、功能扩展与生态共建三个维度持续推进，逐步打造更智能、更普适的代码安全解决方案。以下为具体规划与改进思路：

1. 技术优化与创新

模型轻量化与效率提升

针对当前模型训练资源消耗较高的问题，探索知识蒸馏、模型剪枝等技术，压缩模型体积，降低推理延迟，适配边缘设备（如IDE插件）的实时检测需求。

多模态特征融合增强

引入代码注释、提交历史等开发上下文信息，结合自然语言处理（NLP）技术，提升模型对逻辑漏洞（如权限设计缺陷）的敏感度。

小样本学习与增量训练

开发针对罕见漏洞类型的自适应学习模块，通过小样本学习（Few-shot Learning）解决数据稀疏性问题，并支持在线更新模型，持续适应新型漏洞模式。

2. 功能扩展与场景适配

支持更多编程语言与框架

扩展对Rust、Go等内存安全语言及主流框架（如React、TensorFlow）的检测能力，覆盖云原生、AI模型等新兴领域的安全需求。

开发者交互体验升级

开发交互式代码审查插件（如VS Code扩展），提供“逐行风险提示+修复示例”的沉浸式体验，并集成漏洞知识库（如CWE分类说明），辅助开发者理解风险根源。

自动化修复建议生成

基于强化学习技术，探索从漏洞检测到修复代码生成的闭环能力，例如自动生成补丁代码或推荐安全API替换方案。

3. 生态共建与落地应用

开源社区协作计划

建立漏洞样本众包平台，鼓励开发者提交真实项目漏洞案例，构建动态更新的开放数据集，形成“检测-反馈-优化”的良性循环。

企业级解决方案定制

针对金融、物联网等高安全需求行业，开发定制化规则引擎与合规性检查模块，支持GDPR、等保2.0等法规的自动化审计。

学术与产业联动

联合高校与安全厂商，举办漏洞检测算法竞赛（如基于CTF赛题的数据挑战赛），推动前沿技术的快速落地与产学研融合。

潜在挑战与应对策略

误报率与解释性平衡

在提升检测精度的同时，开发可视化溯源功能（如热力图标记风险代码段），增强模型决策的可解释性，降低开发者信任成本。

对抗性代码防御

研究针对代码混淆、对抗样本攻击的鲁棒性训练方法，确保系统在恶意规避场景下的稳定性。

隐私与合规风险

设计本地化部署方案与数据脱敏机制，满足企业对代码隐私的保护需求，规避开源代码使用的版权风险。



请 老 师 批 评 指 正

T H A N K Y O U F O R W A T C H I N G